A Survey of Multivariate Polynomial Commitment Schemes

Ihyun Nam

Abstract

A commitment scheme is a cryptographic tool that allows one to commit to a hidden value, with the option to open it later at requested places without revealing the secret itself. Commitment schemes have important applications in zero-knowledge proofs and secure multi-party computation, just to name a few. This survey introduces a few multivariate polynomial commitment schemes that are built from a variety of mathematical structures. We study how Orion is constructed using hash functions; Dory, Bulletproofs, and Vampire using the inner-product argument; Signatures of Correct Computation using polynomial factoring; DARK and Dew using groups of unknown order; and Orion+ using a CP-SNARK. For each protocol, we prove its completeness and state its security assumptions.

Contents

1 What is a commitment scheme? 3

2 Multivariate polynomial commitments 3

2.1 Completeness and soundness . . . 3

2.2 Security properties . . . 4

3 Common concepts and notations 4

4 Orion: using hash functions 5

4.1 Correctness and security of Orion . . . 6

5 Dory: using generalized inner-product arguments 7

5.1 Using Dory-PC-RE with EVAL-VMV-RE . . . 7

5.2 Security of Dory . . . 8

6 Bulletproofs: using improved inner-product arguments 8

6.1 Improved inner-product argument . . . 9

6.2 Range proof using Bulletproofs . . . 10

6.3 Completeness and security of Bulletproofs . . . 11

7 Vampire: polynomial commitments combined with inner-product commitments 11

7.1 Completeness of Count . . . 12

7.2 Security assumptions of Vampire . . . 12

8 Signatures of Correct Computation: using polynomial factoring 13

8.1 Correctness and security of SCC . . . 13

9 DARK: using groups of unknown order 14

9.1 Completeness of DARK . . . 15

9.2 Security assumptions of DARK . . . 15

10 Dew: using groups of unknown order 15

10.1 Completeness and security of Dew-PC . . . 17

11 Orion+: using a CP-SNARK 18

11.1 Completeness and security of Orion+ . . . 19

12 Appendix A 20

13 Appendix B 20

1 What is a commitment scheme?

In an interactive protocol involving a prover and a verifier, a commitment scheme is a way in which the prover may prove its knowledge of a statement to the verifier. The simplest way to do so would be for the prover to send the statement directly to the verifier. However, what if the prover wants to keep their knowledge secret or if the statement is simply too large to send directly? Commitment schemes are useful in such cases as they allow the prover to *commit* to the statement and send the resulting commitment instead, which is usually some mathematical structure. Later, the verifier may request the prover to reveal evaluations of the commitment at certain points and provide proof that the claimed evaluations are in fact the correct openings of the commitment. At the end of the interaction, the verifier either accepts or rejects the proof – without actually seeing the statement, ever.

2 Multivariate polynomial commitments

A multivariate polynomial commitment scheme is simply a commitment scheme in which the entity that the prover commits to is a polynomial involving more than one variables. A more formal definition follows.

Definition 1 (Multivariate polynomial commitment scheme)**.** *A multivariate polynomial commitment scheme denoted by* PC *is defined by the following tuple of algorithms.*

- ***PC.Setup***$(\lambda, l, D) \rightarrow$ *(ck, rk). On input a security parameter* λ*, the number of variables* $l \in \mathbb{N}$ *in the polynomial, and a maximum degree bound* $D \in \mathbb{N}$ *of the polynomial, outputs a key pair (ck, rk).*
- ***PC.Commit***$(\phi,$ *ck)*$\rightarrow \mathcal{C}$*. On input an* l*-variate polynomial* ϕ *and ck, outputs a commitment* $\mathcal{C}$ *to* ϕ*.*
- ***PC.Open***$(\phi,$ *ck,* $\vec{x}) \rightarrow \pi$*. On input ck, the polynomial* ϕ*, and an evaluation point* $\vec{x}$ *in* l*-dimension, outputs an evaluation proof* π*.*
- ***PC.Check***$(rk, \mathcal{C}, \vec{x}, \vec{v}, \pi) \rightarrow \{0, 1\}$*. On input rk, commitment* $\mathcal{C}$*, evaluation point* $\vec{x}$*,* $\vec{v}$ *the alleged evaluation of* ϕ *at* $\vec{x}$*, and evaluation proof* π*, outputs 1 if the evaluation proof is correct and 0 otherwise.*

The details of these algorithms may differ in different implementations of a commitment scheme. For example, some schemes have separate online and offline setup stages while some schemes have none. However, in this survey, all commitment schemes are introduced in this generic model for completeness, except for when an algorithm is missing.

2.1 Completeness and soundness

We define the completeness and soundness properties of a (multivariate) polynomial commitment scheme as follows.

Definition 2 (Completeness [1])**.** *A commitment scheme is said to be complete if for every polynomial* ϕ *and evaluation point* $\vec{x}$*, the following probability holds.*

$$Pr\left[PC.Check(\pi, \vec{x}, y, \mathcal{C}) = 1 \;\middle|\; \begin{array}{c} \mathcal{C} \leftarrow PC.Commit(\phi) \\ \vec{x}, y, \pi \leftarrow PC.Open(\phi, \vec{x}, \mathcal{C}) \\ y = \phi(\vec{x}) \end{array}\right] = 1.$$

Completeness states that if a prover is truthful about knowing a statement, then the verifier will *always* accept the prover's proof. The same property is called correctness for argument systems. To define soundness, we first define a negligible function.

Definition 3 (Negligible [2]). *A function $f : \mathbb{N} \to \mathbb{N}$ is said to be negligible if for every $c \in \mathbb{N}$ there exists $k_0 \in \mathbb{N}$ such that for all $k > k_0$, $|f(k)| < k^{-c}$ is true.*

Definition 4 (Soundness). *A commitment scheme is said to be sound if for every polynomial ϕ, an evaluation point $\vec{x}$, proof π, and an efficient adversary $\mathcal{A}$, the following probability holds.*

$$Pr\left[\begin{array}{c} PC.Check(\pi, \vec{x}, \phi(\vec{x}), \mathcal{C}) = 1 \wedge \\ PC.Check(\pi', \vec{x}, \phi(\vec{x})', \mathcal{C}) = 1 \wedge \\ \phi(\vec{x}) \neq \phi(\vec{x})' \end{array} \middle| \begin{array}{c} \mathcal{C} \leftarrow PC.Commit(\phi) \\ (\phi(\vec{x}), \phi(\vec{x})', \pi, \pi') \leftarrow \mathcal{A}(\phi, \vec{x}, \mathcal{C}) \end{array}\right] \leq negl(\lambda).$$

Soundness states that no efficient adversary can produce two distinct valid openings to a commitment at the same point, except for with negligible probability. In other words, if the prover is falsely claiming to know a statement, then the verifier rejects the proof, except for when the prover actually somehow came up with the correct evaluation at the opening without knowing the statement, with negligible probability.

2.2 Security properties

A useful polynomial commitment scheme may also additionally achieve hiding and evaluation binding properties. The *hiding* property states that the distributions of commitments for two distinct messages are computationally indistinguishable. In other words, commitments in a hiding commitment scheme reveal nothing about the committed messages.

Definition 5 (Hiding [3]). *A commitment scheme is said to be hiding if for any probabilistic polynomial time (PPT) adversary $\mathcal{A}$,*

$$\left| Pr\left[b = b' \middle| \begin{array}{c} pp \leftarrow \mathsf{Setup}(1^\lambda) \\ (x_0, x_1) \leftarrow \mathcal{A}(pp), b \xleftarrow{R} \{0,1\}, r \xleftarrow{R} \mathcal{R} \\ \mathcal{C} \leftarrow \mathsf{Commit}(x_b; r), b' \leftarrow \mathcal{A}(pp, \mathcal{C}) \end{array} \right] - \frac{1}{2} \right| \leq negl(\lambda)$$

for security parameter λ.

Evaluation binding states that it must be infeasible for a prover to successfully argue that $f(\vec{z}) = y$ and $f(\vec{z}) = y'$ for a polynomial f, evaluation point $\vec{z}$, and $y \neq y'$.

Definition 6 (Binding [3]). *A commitment scheme is said to be binding if for any PPT adversary $\mathcal{A}$,*

$$Pr\left[\mathsf{Commit}(x_0; r_0) = \mathsf{Commit}(x_1; r_1) \wedge x_0 \neq x_1 \middle| \begin{array}{c} pp \leftarrow \mathsf{Setup}(1^\lambda) \\ x_0, x_1, r_0, r_1 \leftarrow \mathcal{A}(\mathsf{pp}) \end{array}\right] \leq negl(\lambda)$$

for security parameter λ.

This survey states what properties each commitment scheme satisfies, without proving them.

3 Common concepts and notations

We first define some concepts and notations that are used uniformly throughout this survey.

- **Vectors.** Vectors are indicated with an overhead arrow i.e. $\vec{a}$.
- **zk-SNARK.** A zero-knowledge succinct non-interactive argument of knowledge is a cryptographic protocol that allows a prover to prove its knowledge to the verifier without revealing anything about the statement. Many of the commitment schemes discussed in this survey are in fact part of a (zk-)SNARK.
- **Trusted setup.** A setup procedure used to generate a shared secret between a prover and a verifier in a zero-knowledge proof system. A trusted setup destroys the secret after distribution.

- **Prover time, proof size, and verifier time.** These are the three common efficiency majors of a zero-knowledge protocol. *Prover time* is the overhead of the prover to generate the proof, *proof size* is the total communication cost between the prover and the verifier, and *verifier time* is the time the verifier needs to verify evaluations of a commitment [1].

- **Fiat-Shamir transformation.** This is a technique used to make an interactive protocol non-interactive. Informally, instead of having the verifier send a random challenge value to the prover in the Open algorithm, the prover samples a challenge value on its own using a random function.

4 Orion: using hash functions

Orion [1] is a zero-knowledge argument system that achieves $O(N)$ prover time and $O(\log^2 N)$ verifier time based on hash functions. A hash function is a one-way function that maps inputs of arbitrary lengths to hash values of fixed length. While it is easy to compute the hash value of an input, it is computationally difficult to recover an input given its hash value. Orion is plausibly quantum-secure and can be made non-interactive via the Fiat-Shamir transformation.

Orion's novel commitment scheme is based on a linear-time argument presented by Bootle et al. [4] that commits to multilinear polynomials. Most importantly, the commitment scheme performs bit-wise encoding on some message M using the encoding function E_C, the details of which are omitted, and a novel code switching algorithm C_{CS}.

- **PC.Commit**$(\phi) \to \mathcal{R}$. On input a multilinear polynomial ϕ, output a Merkle root commitment $\mathcal{R}$. The coefficients of ϕ is w.
  - Parse w as a $k \times k$ matrix.
  - Compute the tensor code encoding C_1 as a $k \times n$ matrix that encodes each row of M using E_C. Compute C_2 as a $n \times n$ matrix that encodes each column of C_1 using E_C.
  - For $i \in [n]$, compute the Merkle tree root $Root_i =$Merkle.Commit$(C_2[:, i])$.
  - Compute a Merkle tree root $\mathcal{R} =$Merkle.Commit$([Root_0, \cdots, Root_{n-1}])$.

- **PC.Open**$(\phi, \vec{x}, \mathcal{R}) \to c_{\gamma 0}, y_{\gamma 0}, c_1, y_1, C_1[:, idx], Root_{idx}$. On input $\phi, \mathcal{R}$, and an evaluation point $\vec{x}$, the prover additionally has the following values:
  - $\gamma_0 \leftarrow$ a random vector from the verifier in $\mathbb{F}^k$ for some $k \in \mathbb{N}$,
  - $r = r_0 \otimes r_1 \leftarrow$ evaluation point $\vec{x}$ parsed as a tensor product,
  - $y \leftarrow \phi(\vec{x})$, and
  - $\hat{I}, I \leftarrow t \in [n]$ indices randomly sampled by the verifier.

  The prover computes the following:
  - $c_{\gamma 0} = \sum_{i=1}^{k=1} \gamma 0[i] C_1[i]$
  - $y_{\gamma 0} = \sum_{i=0}^{k-1} \gamma 0[i] w[i]$
  - $c_1 = \sum_{i=0}^{k-1} r_0[i] C_1[i]$
  - $y_1 = \sum_{i=0}^{k-1} r_0[i] w[i]$
  - For $idx \in \hat{I}$, compute and send to verifier $C_1[:, idx]$ and the Merkle tree proof of $Root_{idx}$ for $C_2[:, idx]$ under $\mathcal{R}$.

  The prover additionally sends the following to the verifier:
  - $R_{c_1} :=$ Merkle.Commit(c_1)
  - $R_{\gamma_0} :=$ Merkle.Commit(c_{γ_0})

- $y := \langle y_1, r_1 \rangle$
- Output of $C_{CS} : C_2[I[j]], idx], c_2[I[j]], c_{\gamma 0}[I[j]]$ for all $0 < j < |\hat{I}|$ and all $idx \in \hat{I}$
- $\pi \leftarrow$ proof of C_{CS}

- **PC.Check**$(\pi, \vec{x}, y, \mathcal{R}) \rightarrow \{0, 1\}$. On input $\pi, \vec{x}, y$, and $\mathcal{R}$, output 1 if all checks below pass and 0 otherwise. For $idx \in \hat{I}$, check the following:
  - $c_{\gamma 0}[idx] \stackrel{?}{=} \langle \gamma 0, C_1[:, idx] \rangle$
  - $E_C(y_{\gamma 0}) \stackrel{?}{=} c_{\gamma 0}$
  - $c_1[idx] \stackrel{?}{=} \langle r_0, C_1[:, idx] \rangle$
  - $E_C(y_1) \stackrel{?}{=} c_1$
  - $y \stackrel{?}{=} \langle r_1, y_1 \rangle$
  - $E_C(C_1[:, idx])$ consistent with $Root_{idx}$ and its Merkle tree proof.

Code switching is an efficient instantiation of a proof composition technique introduced by Rothblum and Ron-Zewi [5]. Code switching reduces the proof size of the commitment scheme introduced by Bootle et al. [4] and Brakedown [6] from $O(\sqrt{N})$ to $O(\log^2 N)$. Orion calls a pre-defined zero-knowledge argument protocol on C_{CS} and uses the outputs of the algorithm in Check. The code switching algorithm is defined as follows.

The code switching algorithm (C_{CS}) **[1]**

```
Encode c_γ0 := E_C(y_γ0), c_1 := E_C(y_1)
for idx ∈ I do
    Encode C_2[:, idx] := E_C(C_1[:, idx)
    for idx ∈ Î do
        Check c_γ0 ?= ⟨γ0, C_1[:, idx]⟩
        Check c_1[idx] ?= ⟨r_0, C_1[:, idx]⟩
        Check ⟨r_1, y_1⟩ ?= y
        for 0 ≤ j⟨|I| do
            Output c_1[I[j]], c_γ0[I[j]]
            for idx ∈ I do
                Output C_2[I[j], idx]
            end for
        end for
    end for
end for
```

4.1 Correctness and security of Orion

Theorem 1. *Orion is a polynomial commitment that achieves correctness as defined in Definition 2.2.*

Proof. For all $idx \in \hat{I}$, we have $E_C(C_1[:, idx]) = C_2[:, idx]$. PC.Commit computes Merkle.Commit$(E_C(C_1[:, idx]))$ as $Merkle.Commit(C_2[:, idx])$, which is consistent with $Root_{idx}$. The prover sends the Merkle tree proof of $Root_{idx}$ for $C_2[:, idx]$ under $\mathcal{R}$ to verifier, which is correct by construction. □

The security of Orion relies on the hiding property of the Merkle hash function. The Merkle.Commit algorithm is used in Orion to compute the Merkle tree roots $Root_i$ as a commitment to the tensor code encoding of the message M and also to compute the final commitment $\mathcal{R}$. Because the hash algorithm is considered one-way, given an output y to Merkle.Commit (denote this hash function as H) it is infeasible for an adversary to find M such that $y = H(M)$. For simplicity the manipulation of M using E_C and double tensor-encoding is omitted in the statement $H(M)$.

5 Dory: using generalized inner-product arguments

Dory [7] is a transparent setup, public-coin interactive argument for proving completeness of an inner-pairing product. Importantly, Dory works on vector commitments involving elements from two distinct groups. The inner-product argument was initially suggested by Bootle et al. [8] and was used to construct zero-knowledge proofs for arithmetic circuit satisfiability with efficient communication. The inner-product argument takes as inputs two independent generators $G_1, G_2 \in \mathbb{G}^n$, a scalar $c \in \mathbb{Z}_p$, and $P \in \mathbb{G}$. A prover in this argument system wants to convince a verifier that the prover knows $\vec{a}, \vec{b} \in \mathbb{Z}_p^n$ such that

$$\{(G_1, G_2, P, c; \vec{a}, \vec{b}) : P = G_1^{\vec{a}} G_2^{\vec{b}} \wedge c = \langle \vec{a}, \vec{b} \rangle\}.$$

Here, P is the vector commitment to $\vec{a}, \vec{b}$. A full tuple of algorithms for an inner-product commitment scheme is explained in Appendix A.

Definition 7 (Generalized inner-product arguments)**.** *A generalized inner-product argument (GIPA) is a generalization of the inner-product argument. Under GIPA, a prover can additionally prove inner-products between bilinear pairings, not only field elements.*

Additionally, Dory composes Pedersen and AFGHO commitments to commit to matrices using two-tiered homomorphic commitments proposed by Groth [9].

Definition 8 (Pedersen commitment [3])**.** *The Pedersen commitment is a linearly homomorphic scheme that defines a function $M_{pp} \times R_{pp} \to C_{pp}$ for message space $M_{pp} = \mathbb{Z}_p$, randomness space $R_{pp} = \mathbb{Z}_p$, and commitment space $C_{pp} = \mathbb{G}_p$. The* Setup *algorithm outputs $g, h \xleftarrow{R} \mathbb{G}$ and* Commit *outputs $g^x h^r$.*

Definition 9 (AFGHO commitment [10])**.** *Given a commitment key $(v_0, v_1) \in \mathbb{G}_2$ the commitment to $(A_0, A_1) \in \mathbb{G}_1^2$ is given by $e(A_0, v_0)e(A_1, v_1)$.*

5.1 Using Dory-PC-RE with EVAL-VMV-RE

There are several sequential implementations of Dory, each with some more properties than the other. In this survey, we discuss Dory-PC-RE, which is an honest-verifier, statistical zero-knowledge, random evaluation extractable polynomial commitment scheme for r-variable multilinear polynomials, using a commitment to matrices along with the Pedersen commitment. Dory-PC-RE uses EVAL-VMV-RE to evaluate commitments. The resulting commitment scheme achieves linear prover time and logarithmic verifier time.

Take $\mathcal{X} = \mathbb{F}^{n \times m}$ and $M_{ij} \in \mathcal{X}$. Then,

- **Setup**$(1^\lambda) \to$ pp. On input the security parameter λ, output pp $:= (\Gamma_1 \xleftarrow{R} \mathbb{G}_1^m, H_1 \xleftarrow{R} \mathbb{G}_1, \Gamma_2 \xleftarrow{R} \mathbb{G}_2^n, H_2 \xleftarrow{R} \mathbb{G}_2)$
- **Commit**$(\mathsf{pp}; M_{ij}) \to (\mathcal{C}, \mathcal{S})$. On input pp and the ij-th entry in the message M_{ij}, output $(\mathcal{C}, \mathcal{S}) := (\mathcal{C}, (r_{rows}, r_{fin}, \vec{V}))$ for $\begin{cases} r_{rows} \xleftarrow{R} \mathbb{F}^n \\ r_{fin} \xleftarrow{R} \mathbb{F} \\ H_T \leftarrow e(H_1, H_2) \\ V_i \leftarrow \mathsf{Commit}_{\text{Pedersen}}((\Gamma_1, H_1); M_{ij}, r_{rows,i} \\ \mathcal{C} \leftarrow \mathsf{Commit}_{\text{AFGHO}}((\Gamma_2, H_T); \vec{V}, r_{fin}) \end{cases}$
- **Check**$(cc; \mathcal{C}, M, \mathcal{S}) \to \{0, 1\}$. The verifier outputs $\begin{cases} 1 & \text{if } \mathcal{C} = \sum_i \Gamma_{2i} \left(\sum_j M_{ij} \Gamma_{1j} + r_{rows,i} H_1 \right) + r_{fin} \cdot e(H_1, H_2) \\ 0 & \text{if otherwise} \end{cases}$

To see what happens in Dory-PC-RE, we need a bit more information on the Pedersen commitment. For $n = 2^m$, the Pedersen commitment for $\mathbb{F}$ with public parameters $pp_{\mathbb{F}}$, and the matrix commitment for $\mathbb{F}^{n\times n}$ with public parameters $pp_{\mathbb{F}^{n\times n}}$ is such that:

$$\begin{aligned}(\mathcal{C}_M, \mathcal{C}_y, \vec{L}, \vec{R}) &\in \mathcal{L}_{VMV,n,pp_{\mathbb{F}}^{n\times n},pp_{\mathbb{F}}} \subset \mathbb{G}_T \times \mathbb{G}_1 \times \mathbb{F}^n \times \mathbb{F}^n \\ &\Leftrightarrow \exists (M \in \mathbb{F}^{n\times n}, y \in \mathbb{F}, \mathcal{S}_M, \mathcal{S}_y) : y = \vec{L}^T M \vec{R}, \\ &\qquad \text{such that } \mathsf{Open}(\mathsf{pp}_{\mathbb{F}}^{n\times n}, \mathcal{C}_M, M, \mathcal{S}_M) = 1, \mathsf{Open}(\mathsf{pp}_{\mathbb{F}}, \mathcal{C}_y, y, \mathcal{S}_y) = 1.\end{aligned}$$

In Dory-PC-RE, fix Pedersen commitment parameters as $\mathsf{pp}_{\mathbb{F}} = (\Gamma_{1,fin} \xleftarrow{R} \mathbb{G}_1^{2m} \times \mathbb{G}_1)$, and matrix commitment parameters as $\mathsf{pp}_{\mathbb{F}^{n\times n}} = (\Gamma_{1,0} \xleftarrow{R} \Gamma_2^{2m} \times \mathbb{G}_2)$. The prover wants to prove that $(T, y_{com}, \vec{L}, \vec{R}) \in \mathcal{L}_{VMV,n,pp_{\mathbb{F}^{n\times n}},pp_{\mathbb{F}}}$. Here, we have $y_{com} = y\Gamma_{1,fin}$ as the commitment to $y = \vec{L}^T M \vec{R}$ and T as a hiding commitment to $\vec{T}' \in \mathbb{G}_1^n$.

Once the prover computes the vector $\vec{v} = \vec{L}^T M$, we see that by construction $y = \vec{L}^T M \vec{R} = \langle \vec{v}, \vec{R} \rangle$. Also, it follows from the linear homomorphism of Pedersen commitments that

$$v_{com} := \langle \vec{L}, C' \rangle = \mathsf{Commit}_{\Gamma_{1,0}}(\vec{v})$$

is a hiding commitment to $\vec{v}$. Therefore, in EVAL-VMV-RE, it suffices for the prover to prove knowledge of $\vec{T}' \in \mathbb{G}_1^n$ and $\vec{v} \in \mathbb{F}^n$ such that

$$\begin{aligned} T &= \langle \vec{T}', \Gamma_2 \rangle \\ \langle \vec{L}, \vec{T}' \rangle &= \langle \vec{v}, \Gamma_1 \rangle \\ y_{com} &= \langle \vec{v}, \vec{R} \rangle \Gamma_{1,fin}. \end{aligned}$$

We see that this satisfies correctness by construction. Some implementation of Dory also satisfies the Honest-Verifier Statistical Zero-Knowledge properties by achieving hiding.

5.2 Security of Dory

Dory relies on the Symmetric eXternal Diffie-Hellman assumption (SXHD) in the standard model for security.

Definition 10 (SXDH)**.** *For $(\mathbb{F}_p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, G_1, G_2)$ defined as above, the decisional Diffie-Hellman (DDH) assumption holds for $(\mathbb{F}_p, \mathbb{G}_1, G_1)$ and $(\mathbb{F}_p, \mathbb{G}_2, G_2)$. In any group, DDH implies DLOG, double pairing assumption, and reverse double pairing assumption defined below.*

Definition 11 (Double pairing assumption [7])**.** *For $(\mathbb{F}_p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, G_1, G_2)$ defined as above, given $A_1, A_2 \xleftarrow{R} \mathbb{G}_1$ no non-uniform PPT adversary can compute non-trivial $B_1, B_2 \in \mathbb{G}_2$ such that $A_1B_1 + A_2B_2 = 0$. Similarly, given $A_1, A_2 \xleftarrow{R} \mathbb{G}_2$ no adversary can compute non-trivial $B_1, B_2 \in \mathbb{G}_1$ such that $B_1A_1 + B_2A_2 = 0$.*

Definition 12 (Reverse double pairing assumption[7])**.** *For $(\mathbb{F}_p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, G_1, G_2)$ defined as above and $n = poly(\lambda)$, given $\vec{A} \xleftarrow{R} \mathbb{G}_1^n$ no non-uniform PPT adversary can compute a non-trivial $\vec{B} \in \mathbb{G}_2^n$ such that $\langle \vec{A}, \vec{B} \rangle = 0$. Similarly, given $\vec{A} \xleftarrow{R} \mathbb{G}_2^n$, no adversary can compute non-trivial $\vec{B} \in \mathbb{G}_2^1$ such that $\langle \vec{B}, \vec{A} \rangle = 0$.*

6 Bulletproofs: using improved inner-product arguments

Bulletproofs [3] is a non-interactive zero-knowledge proof protocol that does not require a trusted setup. While Bulletproofs can be used to prove a variety of statements on a committed value, this survey focuses on how Bulletproofs can yield a range proof. That is, proving that a commitment V contains a number v that lies in some range, without revealing v. The proof size is logarithmic in the size of the witness. Bulletproofs have especially important applications to secret monetary transactions on distributed and trustless blockchains.

6.1 Improved inner-product argument

This is a modification of the inner-product argument introduced by Bootle et al.[4]. Bulletproofs build a proof system for the relation:

$$\{(\vec{g}, \vec{h}, u, P; \vec{a}, \vec{b} \in \mathbb{Z}_p^n) : P = \vec{g}^{\vec{a}} \vec{h}^{\vec{b}} \cdot u^{\langle \vec{a}, \vec{b} \rangle}\} \qquad (*).$$

This is a special case of the original inner-product argument when $c = \langle \vec{a}, \vec{b} \rangle$ is given as part of the vector commitment P. Bulletproofs construct a proof protocol for (*) with $2\log_2(n)$ proof size, as opposed to a naive solution with $2n$ proof size when the prover directly sends $\vec{a}, \vec{b} \in \mathbb{Z}_p^n$ to the verifier. Importantly for $n > 1$, Open function, Bulletproofs have the prover and the verifier engage in an following recursive inner-product argument for P'.
To see how, first define a hash function $H : \mathbb{Z}_p^{2n+1} \to \mathbb{G}$ that takes in as inputs $\vec{a}, \vec{a}', \vec{b}, \vec{b}' \in \mathbb{Z}_p^{n'}$ and $c \in \mathbb{Z}_p$, and outputs

$$H(\vec{a}, \vec{a}', \vec{b}, \vec{b}, c) = g_{[:n']}^{a} \cdot g_{[:n']}^{a'} \cdot h_{[:n']}^{b} \cdot h_{[:n']}^{b'} \cdot u^c \in \mathbb{G}.$$

Here, the notation $\vec{a}_{[:n]}$ denotes slices of vectors $(a_1, \cdots, a_n) \in \mathbb{F}^n$. Bulletproofs implement an *improved* inner-product argument that reduces the overall proof size by a factor of 3. The following tuple of algorithms defines the relation $(*)$, and therefore also the proof system for Bulletproofs.

- Step 1. Prover computes the following and send L, R to the verifier.
  - $n' = \frac{n}{2}$
  - $c_L = \langle \vec{a}_{[:n']}, \vec{b}_{[n':]} \rangle$
  - $c_R = \langle \vec{a}_{[n':]}, \vec{b}_{[:n']} \rangle$
  - $L = \vec{g}_{[n':]}^{\vec{a}_{[:n']}} \vec{h}_{[:n']}^{\vec{b}_{[n':]}} u^{c_L}$
  - $R = \vec{g}_{[:n']}^{\vec{a}_{[n':]}} \vec{h}_{[n':]}^{\vec{b}_{[:n']}} u^{c_R}$
- Step 2. Verifier randomly samples x from $\mathbb{Z}_p^\star$ and sends it to the prover.
- Step 3. Now that they both have x, the prover and the verifier compute
  - $\vec{g}' = \vec{g}_{[:n']}^{x^{-1}} \circ \vec{g}_{[n':]}^{x}$
  - $\vec{h}' = \vec{h}_{[:n']}^{x} \circ \vec{h}_{[n':]}^{x^{-1}}$
  - $P' = L^{x^2} P R^{x^{-2}}$
- Step 4. Prover computes
  - $\vec{a}' = \vec{a}_{[:n']} \cdot x + \vec{a}_{[n':]} \cdot x^{-1}$
  - $\vec{b}' = \vec{b}_{[:n']} \cdot x^{-1} + \vec{b}_{[n':]} \cdot x$
- Step 5. Verifier outputs 1 if

  $$P' = \left(\vec{g}_{[:n']}^{x^{-1}} \circ \vec{g}_{[n':]}^{x} \right)^{\vec{a}'} \cdot \left(\vec{h}_{[:n']}^{x} \circ \vec{h}_{[n':]}^{x^{-1}} \right)^{\vec{b}'} \cdot u^{\langle \vec{a}', \vec{b}' \rangle}$$

  and 0 otherwise.
- Step 5. Repeat the protocol recursively on input $(\vec{g}', \vec{h}', u, P'; \vec{a}', \vec{b}')$ so that the prover ends up sending $(L_1, R_1), \cdots, (L_{\log_2 n}, R_{\log_2 n}), (a, b)$.

6.2 Range proof using Bulletproofs

We introduce an example of a range proof using Bulletproofs using insights from a Pedersen commitment. Curious readers should read the original Bulletproofs paper to see how they can also be used to make zero-knowledge proofs for arithmetic circuits. The proof system for a range proof proves the following relation:

$$(g, h \in \mathbb{G}, V, n; v, \gamma \in \mathbb{Z}_p : V = h^\gamma g^v \wedge v \in [0, 2^{-1}]) \qquad (*)$$

for a group $\mathbb{G}$ and a Pedersen commitment $V \in \mathbb{G}$ to $v \in [0, 2^n - 1]$ with randomness γ.

- **Setup**(a_L, a_R)
- **Commit**$(\vec{a}_L, \vec{a}_R) \to A$**.** On input vectors $\vec{a}_L \in \{0,1\}^n$ such that $\langle \vec{a}_L, \vec{2}^n \rangle = v$ and $\vec{a}_R = \vec{a}_L - \vec{1}^n \in \mathbb{Z}_p^n$, output the commitment $A = h^\alpha \vec{g}^{\vec{a}_L} \vec{h}^{\vec{a}_R} \in \mathbb{G}$ for α randomly sampled from $\mathbb{Z}_p$. Also randomly sample 'blinding vectors' $\vec{s}_L, \vec{s}_R$ from $\mathbb{Z}_p^n$ and ρ from $\mathbb{Z}_p$. Compute $S = h^\rho \vec{g}^{\vec{s}_L} \vec{h}^{\vec{s}_R} \in \mathbb{G}$. Send A, S to the verifier.
- **Open**(y, z). Verifier randomly samples $y, z \in \mathbb{Z}_p$ and sends them to the prover. At this point, the protocol defines the following polynomials:
  - $l(X) = (\vec{a}_L - z \cdot \vec{1}^n) + \vec{s}_L \cdot X \in \mathbb{Z}_p^n[X]$
  - $r(X) = \vec{y}^n \circ (\vec{a}_R + z \cdot \vec{1}^n + \vec{s}_R \cdot X) + z^2 \cdot \vec{2}^n \in \mathbb{Z}_p^n[X]$
  - $t(X) = \langle l(X), r(X) \rangle = t_0 + t_1 \cdot X + t_2 \cdot X^2 \in \mathbb{Z}_p[X]$

  The prover's job is reduced to convincing the verifier that $t_0 = v \cdot z^2 + \delta(y, z)$ where $\delta(y, z) = (z - z^2) \cdot \langle \vec{1}^n, \vec{y}^n \rangle - z^3 \langle \vec{1}^n, \vec{2}^n \rangle \in \mathbb{Z}_p$. To do so, it goes through another cycle of **Commit**$_2$ and **Open**$_2$.
- **Commit**$_2(t_1, t_2) \to T_1, T_2$**.** On input t_1, t_2 and τ_1, τ_2 sampled randomly from $\mathbb{Z}_p$, output the commitments $T_i = g^{t_i} h^{\tau_i}$ for $i = \{1, 2\}$. Prover sends T_1, T_2 to the verifier.
- **Open**$_2(x) \to \tau_x, \mu, \hat{t}, \vec{l}, \vec{r}$. On input a random challenge $x \in \mathbb{Z}_p^\star$ from the verifier, the prover outputs the evaluation of l and r at x along with a few others as follows.
  - $\vec{l} = l(x) = \vec{a}_L - z \cdot \vec{1}^n + \vec{s}_L \cdot x$
  - $\vec{r} = r(x) = \vec{y}^n \circ (\vec{a}_R + z \cdot \vec{1}^n + \vec{s}_R \cdot x) + z^2 \cdot \vec{2}^n$
  - $\hat{t} = \langle \vec{1}, \vec{r} \rangle$
  - $\tau_x = \tau_2 \cdot x^2 + \tau_1 \cdot x + z^2 \cdot \gamma$
  - $\mu = \alpha + \rho \cdot x$
- **Check**$(\vec{l}, \vec{r}, \tau_x, \mu, \hat{t}) \to \{0, 1\}$. On input $\vec{l}, \vec{r}, \tau_x, \mu, \hat{t}$, the verifier outputs 1 if all the checks below pass and 0 otherwise.
  - $g^{\hat{t}} h^{\tau_x} \stackrel{?}{=} V^{z^2} \cdot g^{\delta(y,z)} \cdot T_1^x \cdot T_2^{x^2}$
  - $A \cdot S^x \cdot \vec{g}^{-z} \cdot (\vec{h'})^{z \cdot \vec{y}^n + z^2 \cdot \vec{2}^n} \stackrel{?}{=} h^\mu \cdot \vec{g}^{\vec{l}} \cdot (\vec{h'})^{\vec{r}}$
  - $\hat{t} \stackrel{?}{=} \langle \vec{l}, \vec{r} \rangle$

This can be made into a non-interactive proof through Fiat-Shamir transformation and additionally achieve knowledge soundness.

6.3 Completeness and security of Bulletproofs

Theorem 2. *Bulletproofs achieves completeness as defined in Definition 2.2.*

Proof. We want to show that $L^{(x^2)} \cdot P \cdot R^{(x^2)} \stackrel{?}{=} H(x^{-1}\vec{a'}, x\vec{a'}, x\vec{b'}, x^{-1}\vec{b'}, \langle \vec{a'}, \vec{b'} \rangle)$ in Check holds for an honest prover. The LHS is equal to $L^{x^2} \cdot P \cdot R^{x^{-2}}$, which is equal to

$$H(\vec{a}_{[:n']} + x^{-2}\vec{a}_{[n':]}, x^2\vec{a}_{[:n']} + \vec{a}_{[n':]}, x^2\vec{b}_{[n':]} + \vec{b}_{[:n']}, \vec{b}_{[n':]} + x^{-2}\vec{b}_{[:n']}, \langle \vec{a'}, \vec{b'} \rangle).$$

Using the properties of H as defined, the RHS is equal to $H(x^{-1}\vec{a'}, x\vec{a'}, x\vec{b'}, x^{-1}\vec{b'}, \langle \vec{a'}, \vec{b'} \rangle)$ as required. □

Bulletproofs also provides the arithmetic circuit protocol. This can be used with the improved inner-product argument to achieve perfect completeness and computational soundness under the discrete logarithm assumption. The security of Bulletproofs is reduced to the hardness of the discrete logarithm problem, which guarantees the perfectly hiding and computationally binding properties of Pedersen commitments.

Definition 13 (Discrete logarithm problem[11])**.** *Given a group G, a generator g of the group and an element h of G, find the discrete logarithm to the base g of h in the group G.*

The discrete logarithm problem may or may not be hard. The Discrete logarithm relation that Bulletproofs relies on, however, is a special case when $n \leq 1$ and is considered hard.

Definition 14 (Discrete Logarithm Relation [3])**.** *For all PPT adversaries $\mathcal{A}$:*

$$Pr\left[\exists a_i \neq 0 \wedge \Pi_{i=1}^{n} g_i^{a_i} = 1 \;\middle|\; \begin{array}{c} \mathbb{G} = Setup(1^\lambda) \\ g_1, \cdots, g_n \xleftarrow{R} \mathbb{G} \\ a_1, \cdots, a_n \in \mathbb{Z}_p \leftarrow \mathcal{A}(G, g_1, \cdots, g_n) \end{array}\right] \leq \mu(\lambda).$$

7 Vampire: polynomial commitments combined with inner-product commitments

Lipmaa et al. present Vampire [12]: a pairing-based updatable and universal zk-SNARK. The key subroutine that Vampire uses for its commitment component is Count, a novel univariate sumcheck argument. Informally, a sumcheck argument is a sumcheck protocol that is used to succinctly prove knowledge of openings for certain commitments [13]. Commitment schemes that allow this are called sumcheck-friendly. In a univariate sumcheck argument for multiplicative subgroups, given a finite field $\mathbb{F}$ and some multiplicative subgroup $\mathbb{H} \subset \mathbb{F}$, the prover convinces the verifier that the committed polynomial $f \in \mathbb{F}$ sums to the agreed upon value $v \in \mathbb{F}$ over $\mathbb{H}$.

Count improves on the online efficiency of the previously optimal Aurora [14]. In Count, the prover only needs to outputs one ILV commitment [15] (which is essentially a single group element) instead of two polynomial commitments (two group elements) in Aurora. More formally, the commitment scheme in Count is defined by the following tuple of algorithms. It has three separate Setup phases called PGen, KGen, and Derive.

- **PGen**$(1^\lambda) \rightarrow$ pp$=(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$. On input the security parameter λ, PGen is a bilinear group generator that returns a prime p, three additive cyclic groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ of order p, a non-degenerate efficiently computable bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, a generator g_b for $\mathbb{G}$.
- **KGen**(pp$, n_h, d, d_{gap}) \rightarrow$ srs. On input pp, $n_h = |\mathbb{H}|, d$ the degree of the commitment polynomial $f \in \mathbb{F}[X]$, and $d_{gap} \geq n_h \cdot \lfloor d/n_h \rfloor$, compute the following.
  - $S_1(X) \leftarrow \{(X^i)_{i=0:i\neq d_{gap}}^{d_{gap}+d}\}$

- $S_2(X) \leftarrow \{1, X, (X^{d_{gap}-n_h i})_{i=0}^{\lfloor d/n_h \rfloor}\}$
- $\sigma \xleftarrow{R} \mathbb{F}^*$

Finally output $\mathsf{srs} \leftarrow (p, n_h, d, d_{gap}, [g(\sigma) : g \in S_1(X)]_1, [g(\sigma) : g \in S_2(X)]_2)$.

- **Derive**(srs) $\rightarrow \mathsf{ek}_{\mathcal{R}}$, $\mathsf{vk}_{\mathcal{R}}$. On input srs, compute $S(X) \leftarrow \sigma_{i=0}^{\lfloor d/n_h \rfloor} X^{d_{gap}-n_h i}$. Set $\mathsf{ek}_{\mathcal{R}} \leftarrow$srs and set $\mathsf{vk}_{\mathcal{R}} \leftarrow$(srs,$[S(\sigma)]_2, [\sigma^{d_{gap}}]_T$). Return ($\mathsf{ek}_{\mathcal{R}}$, $\mathsf{vk}_{\mathcal{R}}$).

Once the preprocessing is done, the prover has $(\mathsf{ek}_{\mathcal{R}}, ([f(\sigma)]_1, v_f), f)$, and the verifier has $(\mathsf{ek}_{\mathcal{R}}, ([f(\sigma)]_1, v_f))$. The prover sends to the verifier the commitment to

$$[f_{ipc}(\sigma)]_1 = f(X)S(X) - v_f/n_h \cdot X^{d_{gap}}.$$

In the Check algorithm, the verifier accepts if and only if

$$[f(\sigma)]_1 \bullet [S(\sigma)]_2 - [f_{ipc}(\sigma)]_1 \bullet [1]_2 = v_f/n_h \cdot [\sigma^{d_{gap}}]_T \qquad (*).$$

Notice that $(*)$ is a direct adaptation of the generic evaluation algorithm in the inner-product commitment scheme (Appendix A) where instead of committing to a vector μ the prover is committing to a multivariate polynomial f.

7.1 Completeness of Count

To see why (*) correctly checks the validity of evaluations, we first define a lemma. A full proof for this lemma is present in the original Vampire paper [12].

Lemma 1. *We have the following set up.*

- $\mathbb{H}$*: order* $n_h > 1$ *multiplicative subgroup of* $\mathbb{F}^*$
- d_{gap}*: fixed parameter by Vampire such that* $d_{gap} \leq n_h \cdot \lfloor d/n_h \rfloor$ *for* $d > 0$
- $f \in PolyPunc(d, d_{gap}, X) := \{f(X) = \sum_{i=0}^{d_{gap}+d} f_i X^i \in \mathbb{F}_{\leq d_{gap}+d}[X] : f_{d_{gap}} = 0\}$
- $S(X) := \sum_{i=0}^{\lfloor d/n_h \rfloor} X^{d_{gap}-n_h i} \in \mathbb{F}_{\leq d_{gap}}[X]$

Then, $\sum_{\chi \in \mathbb{H}} f(\chi) = v_f$ *and* $deg(f) \leq d$ *if and only if there exists* $f_{ipc} \in PolyPunc_{\mathbb{F}}(d, d_{gap}, X)$ *such that*

$$f(X)S(X) - f_{idc}(X) = \frac{v_f}{n_h} \cdot X^{d_{gap}}. \qquad (\star)$$

Then, (*) is a natural manipulation of $(\star)$ and thus completeness follows directly from Lemma 7.1.

7.2 Security assumptions of Vampire

Count relies on the $(d_1, d_2) - PDL$ (Power Discrete Logarithm) assumption of KGen.

Definition 15 ($(d_1, d_2) - PDL$ assumption [12])**.** *Let* $d_1(\lambda), d_2(\lambda) \in$ *Poly*(λ)*.* *KGen is* $(d_1, d_2) - PDL$ *secure if for any non-uniform PPT* $\mathcal{A}$*,*

$$Adv^{PDL}_{d_1,d_2,KeyGen,\mathcal{A}}(\lambda) := Pr\left[\mathcal{A}\left(p, [(x^i)_{i=0}^{d_1}]_1, [(x^i)_{i=0}^{d_2}]_2\right) = x \middle| p \leftarrow \mathsf{KGen}\left(1^\lambda; x \xleftarrow{R} \mathbb{F} *\right)\right]$$
$$= negl(\lambda).$$

Count not only reduces the communication cost of Aurora but also gets rid of a low-degree test, which the prover in Aurora uses to convince the verifier of the degree bound of some polynomial R that is part of a linear combination for f.

8 Signatures of Correct Computation: using polynomial factoring

Signatures of correct computation (SCC) [16] is a model to verify dynamic computations by outsourcing the function to evaluate to an untrusted server. An SCC scheme for multivariate polynomial evaluation is equivalent to a publicly verifiable multivariate polynomial commitment scheme. SCC relies on polynomial decomposition, which is an extension of the factoring theorem for univariate polynomials, as defined below.

Definition 16 (Polynomial decomposition)**.** *Let $f(\vec{x}) \in \mathbb{Z}_p[\vec{x}]$ be an n-variate polynomial. For all $\vec{a} \in \mathbb{Z}_p^n$, there exist polynomials $q_i(\vec{x}) \in \mathbb{Z}_p[\vec{x}]$ such that the polynomial $f(\vec{x}) - f(\vec{a})$ can be expressed as $f(\vec{x}) - f(\vec{a}) = \sum_{i=1}^{n}(x_i - a_i)q_i(\vec{x})$. There exists a polynomial-time algorithm that finds such $q_i(\vec{x})$.*

MC_m is a commitment scheme built by Lehmkuhl and Chiesa [17] based on SCC that is additionally hiding. MC_m commits to polynomials with a single degree bound and is a direct generalization of the polynomial commitment scheme in KZG [18]. MC_m is defined by the following tuple of algorithms when committing to an l-variate polynomial p with maximum degree bound $D \in \mathbb{N}$. Furthermore, $\mathcal{W}_{l,D}$ is a set of all multisets of $\{1, \cdots, D\}$ with cardinality at most D. Notice that polynomial decomposition is used in Open when the prover (*source* in SCC) produces the evaluation point for the requested opening.

- $MC_m.\mathsf{Setup}(\lambda, l, D) \to (\mathsf{ck},\ \mathsf{rk})$. Sample a bilinear group $\langle \text{group} \rangle \leftarrow \mathsf{SampleGrp}(1^\lambda)$. Parse $\langle \text{group} \rangle$ as a tuple $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, q, G_1, G_2, e)$. Sample random elements $\beta_a, \cdots, \beta_l$ from $\mathbb{F}_q$. Then, $\mathsf{ck} := (\langle \text{group} \rangle, \sum)$ and $\mathsf{rk} := (D, \langle \text{group} \rangle, \beta_1 G_1, \cdots, \beta_l G_1)$ for $\sum = \{(\Pi_{i \in W} \beta_i)G\}_{W \in W_{l,D}} \in \mathbb{G}_1^{D^l}$.
- $MC_m.\mathsf{Commit}(\mathsf{ck},\ p) \to \mathcal{C}$. Compute the commitment $\mathcal{C} = [c_i]_i^n$ for $c_i := p_i(\beta)G$.
- $MC_m.\mathsf{Open}(\mathsf{ck},\ p, \vec{x}, \xi) \to \pi$. For an opening challenge ξ and evaluation point $\vec{x} \in \mathbb{F}_q^l$, compute the linear combination of polynomials $p(\vec{X}) := \sum_{i=1}^{n} \xi^i p_i(\vec{X})$. Compute $\vec{w} := [w_j]_{j=1}^l$ such that
$$p(\vec{X}) - p(\vec{x}) = \sum_{j=1}^{l}(X_j - z_j)w_j(\vec{X}).$$
Compute evaluation proof as $\pi := [w_j \beta G]_{j=1}^l$.
- $MC_m.\mathsf{Check}(\mathsf{rk}, \mathcal{C}, \vec{x}, v, \pi) \to \{0, 1\}$. For the opening challenge ξ, compute $C := \sum_{i=1}^{n} \xi^i c_i$ and $v := \sum_{i=1}^{n} \xi^i v_i$. Output 1 if the following equality holds and 0 otherwise:
$$e(C - vG, H) = \Pi_{j=1}^{l} e(w_j(\beta)G, \beta_j H - z_j H).$$

8.1 Correctness and security of SCC

Correctness of MC_m follows from polynomial decomposition and the bilinear properties of e.

Theorem 3. *The commitment scheme MC_m achieves correctness as defined in Definition 2.2.*

Proof. This proof was directly adopted from the original paper with little modification. Let an efficient adversary $\mathcal{A}(\mathsf{ck},\mathsf{rk})$ choose l-variate polyomials $\vec{p} := [p_i]_{i=1}^n \in \mathbb{F}_q$, evaluation point $\vec{x}$, and

opening challenge ξ. For $\vec{v} = \vec{p}(\vec{x})$, observe that

$$\begin{aligned} e(C - vG, H) &= e(\sum_{i=1}^{n} \xi^i c_i - (\sum_{i=1}^{n} \xi^i v_i)G, H) \\ &= e((p(\beta) - p(\vec{x}))G, H) \\ &= e((\sum_{j=1}^{l} (\beta_j - x_j) w_i(\beta))G, H) \\ &= \Pi_{j=1}^{l} e((\beta_j - x_j)(w_j(\beta))G, H) \\ &= \Pi_{j=1}^{l} e(w_j, \beta_j H - x_j H) \end{aligned}$$

as required by Check [17]. □

The security of SCC relies on the following computational assumption.

Definition 17 (Bilinear l-strong Diffie-Hellman assumption [19])**.** *Suppose λ is the security parameter and let $(p, \mathbb{G}, \mathbb{G}_T, e, g)$ be a uniformly randomly generataed tuple of bilinear pairings parameters. Given the elements $g, g^t, \cdots, g^{t^l} \in \mathbb{G}$ for some t chosen at random from $\mathbb{Z}_p^*$ and for $l = poly(\lambda)$, there is no PPT algorithm that can output the pair $(c, e(g,g)^{1/(t+c)}) \in \mathbb{Z}_p^*/-t/ \times \mathbb{G}_T$ except with negligible probability $negl(\lambda)$.*

9 DARK: using groups of unknown order

Bunz et al. [20] built a polynomial commitment scheme based on Diophantine Argument of Knowledge (DARK) compilers. We conveniently refer to the resulting polynomial commitment scheme as DARK. DARK does not require a trusted setup and thus is used to build a new transparent proof system called Supersonic.

DARK is an instantiation of an abstract homomorphic commitment function $[\![*]\!] : \mathbb{Z}_p[X] \to \mathbb{S}$ for some prime p and some set $\mathbb{S}$. The following tuple of algorithms is a concrete implementation of $[\![*]\!]$. Importantly, the Check algorithm recursively reduces the degree of f from d to $\lfloor \frac{n}{2} \rfloor$. At every recursive step, the verifier holds a commitment $[\![f(X)]\!]$ to $f(X)$, and the prover is bound to the underlying polynomial. The key to DARK is that the order of $\mathbb{G}$ cannot be computed efficiently – hence the name group of unknown order.

- **DARK.Setup** $(1^\lambda) \to$pp. On input the security parameter λ, sample $\mathbb{G} \xleftarrow{R} GGen(\lambda)$ and $G \xleftarrow{R} \mathbb{G}$, output pp as $(\lambda, \mathbb{G}, G, q)$ where q is a large integer.
- **DARK.Commit**(pp, $f(X)$) $\to (\mathcal{C}; f(X), \hat{f}(X))$. On input pp and the commitment polynomial $f(X)$, compute and return $C = \hat{f}(q) \cdot G$, $f(X)$, and $f(\hat{X})$ is $f(X)$ "lifted" to an integer polynomial with bounded coefficients such that $f(\hat{X}) \mod p = f(X)$.
- **DARK.Open**(pp, $C, f(X), \hat{f}(X)$). Check that $\hat{f}(X) \in \mathbb{Z}(q/2)[X], \hat{f}(q) \cdot G = C$ and $f(X) = \hat{f}(X) \mod p$.
- **DARK.Check**. The prover computes $C_L = \hat{f}_L(q, q^2, \cdots, q^{(2^{\mu'}-1)}) \cdot G$ and $C_R = \hat{f}_R(q, q^2, \cdots, q^{(2^{\mu'}-1)}) \cdot G$. Then, the verifier outsources the expensive checking of $C_L + q^{d'+1} \cdot C_R = C$ to the prover by using a proof of exponentiation protocol [21].

When used with a μ-variate polynomial of degree d, the proof size and verification work for DARK is $\mu \log(d)$. That is, the prover and verifier times are logarithmic in size to the number of coefficients in the commitment polynomial.

9.1 Completeness of DARK

Theorem 4. *The commitment scheme DARK achieves completeness for μ-linear polynomials in $\mathbb{Z}_p[X]$ as defined in Definition 2.2.*

Proof. This proof was directly adopted from the original paper with little modification. Proving the completeness of DARK can be reduced to proving that $b < \frac{q}{2}$ and $|f| \leq b$. We show by induction that for each recursive call of the evaluation protocol, inputs satisfy the following constraints: $f(X_1, \cdots, X_\mu)$ is μ-linear, $C = G^{f(\vec{q})}, f(X) \in \mathbb{Z}(b)$, and $f(z_1, \cdots, z_\mu) = y \mod p$.

In the first iteration of the evaluation protocol, the prover maps the coefficients of a polynomial $\tilde{f}(X_1, \cdots, X_\mu) \in \mathbb{Z}_p$ to a μ-linear polynomial $f(X_1, \cdots, C_\mu)$ with coefficients in $\mathbb{Z}(p-1)$. In the final iteration of the evaluation protocol, the prover produces f, such that $|f| < b$ by construction.

Then, in the i-th iteration of the protocol, the prover computes $(i-1)$-linear polynomials f_L and f_R such that $f_L(X_1, \cdots, X_{i-1}) + X_i f_R(X_1, \cdots, X_{i-1}) = f(X_1, \cdots, X_i)$. It follows that $f(z_1, \cdots, z_i) \mod p = f_L(z_1, \cdots, z_{i-1}) + z_i f_R(z_1, \cdots, z_{i-1}) \mod p = y_L + z_i y_R \mod p = y$. Finally, the prover can construct an $(i-1)$-linear polynomial $f' = f_L + \alpha f_R \in \mathbb{Z}(2^\lambda b)$. This computes the correct values of f, y, and b such that $|f| < b$ [20].

This proof can be applied to μ-variate polynomials by setting f as such and adjusting the coefficients space accordingly. □

9.2 Security assumptions of DARK

The security of DARK depends on the order, adaptive root, and r-fractional root assumptions defind below.

Definition 18 (Order assumption [22])**.** *For a group of unknown order $\mathbb{G} \leftarrow GGen(k)$, the Order assumption holds if for any adversary $\mathcal{A}$*

$$Pr\left[w \neq 1 \wedge w^\alpha = 1 \;\middle|\; \begin{array}{c} GGen(k) \to \mathbb{G} \\ \mathcal{A}(\mathbb{G}) \to (w, \alpha) \\ where\ |\alpha| < 2^{poly(k)} \in \mathbb{Z}\ and\ w \in \mathbb{G} \end{array}\right] \leq negl(k).$$

The order assumption is implied by the adaptive root assumption defined below.

Definition 19 (Adaptive root assumption [23])**.** *For a group of unknown order $\mathbb{G} \leftarrow \mathsf{GGen}$, we say that the Adaptive root assumption holds for GGen if for any efficient adversaries $\mathcal{A}_0, \mathcal{A}_1$:*

$$Pr\left[z^l = w \neq 1 \;\middle|\; \begin{array}{c} \mathsf{GGen}(k) \to \mathbb{G} \\ \mathcal{A}_0(\mathbb{G}) \to (w, state) \\ l \xleftarrow{R} Primes(k) \\ \mathcal{A}_1(l, state) \to z \end{array}\right] \leq negl(k).$$

Definition 20 (r-Fractional Root Assumption)**.** *For a group of unknown order, the r-Fractional Root Assumption holds for GGen if for any efficient adversary $\mathcal{A}$:*

$$Pr\left[u^\beta = g^\alpha \wedge \frac{\beta}{gcd(\alpha,\beta)} \neq r^k, k \in \mathbb{N} \;\middle|\; \begin{array}{c} \mathsf{GGen}(k) \to \mathbb{G} \\ \mathbb{G} \to g \\ \mathcal{A}(\mathbb{G}, g) \to (\alpha, \beta, u) \\ where\ |\alpha| < 2^{poly(k)}, \\ |\beta| < 2^{poly(k)} \in \mathbb{Z}\ and\ u \in \mathbb{G} \end{array}\right] \leq negl(k).$$

10 Dew: using groups of unknown order

Arun et al. introduces Dew-PC, a new polynomial commitment scheme with constant size evaluation proofs and logarithm verifier time based on a transparent inner-product commitment scheme.

This commitment scheme is ultimately used to compile an information-theoretic proof system and yield Dew [24]: a transparent (meaning no trusted setup) constant-sized zk-SNARK. Dew-PC is defined by the following tuple of algorithms.

- **Setup**$(1^\lambda, D) \rightarrow$ pp=$(\lambda, \mathbb{G}, g, PRG, p)$. On input k the security parameter and D the maximum degree of the commitment polynomial, output:
  - $\mathbb{G} \leftarrow \mathsf{GGen}(\lambda)$
  - $g \xleftarrow{\$} \mathbb{G}$
  - A pseudorandom generator PRG: $\{0,1\}^\lambda \rightarrow \mathbb{Z}_p^{2\log(D+1)}$
  - A large prime p such that $len(p) = poly(\lambda)$
- **Commit**(pp,$D, f(X), l-1) \rightarrow (\mathcal{C}, \mathbf{f})$. On input public parameter pp, a degree (l-1) polynomial f(X) in $\mathbb{Z}_p[X]$, output:
  - The commitment $\mathcal{C} := g^{\sum i=0^{l-1} f_i \alpha^{2i}}$
  - The coefficient vector $\mathbf{f}$ where f_i's are the coefficients of the polynomial
- **Open**(pp,$D, f(X), l-1, C, \tilde{\mathbf{f}}) \rightarrow$ pp=$(\lambda, \mathbb{G}, g, PRG, p)$. Output 1 if all of the following checks pass, and 0 otherwise.
  - $l-1 \leq D$
  - $\sum_{i=0}^{l-1} \tilde{f}_i \alpha^{2i} \in \mathbb{Z}$
  - $\mathcal{C} = g^{\sum_{i=0}^{l-1} \tilde{f}_i \alpha^{2i}}$
  - $\tilde{\mathbf{f}} \in \{\frac{a}{b} | gcd(a,b) = gcd(b,p) = 1, 0 < b < p^{1+\log l}, |a/b| \leq (2l+1)\alpha\}^l$
  - $\tilde{f}_i = f_i \mod p$
- **Check**(pp,$D, C, l-1, x, v, f(X)) \rightarrow$ **logTEST**$(\mathcal{C}, l-1; f(X))$, **logIPP**$(\mathcal{C}, l-1, x, v; f(X))$. Return 1 if both subprocesses **logTEST** and **logIPP** accept. In both protocols described below, PoKPE is an argument of knowledge for the relation R_{PoKPE} in the generic group model. It is reproduced in Appendix A.
  - **logTEST:** Replace the query vector $\mathbf{z}$ in Figure 1 with

    $$z_k \equiv z_{k_0, \cdots, k_{\log l - 1}} := \Pi_{j=1}^{\log l} x_{j,k_{j-1}}.$$

    Here, $x_1, \cdots, x_{\log l}$ are sampled from $\mathbb{Z}_p^2$ where $x_j = (x_{j,0}, x_{j,1})$, and $(k_0, \cdots, k_{\log l - 1})$ is the base-2 representation of $0 \leq k \leq l-1$.
  - **logIPP:** Replace the query vector $\mathbf{q}$ in Figure 2 with

    $$q_k := \Pi_{0 \leq j \leq \log l - 1}(x^{k_j 2^j} \mod p)$$

    where $x \in \mathbb{Z}_p$.

TEST

Prover		**Verifier**
$C := \mathsf{commit}(\mathsf{pp}, \mathbf{f} \in \mathbb{Z}_p^l)$	$\xrightarrow{C}$	
	$\xleftarrow{\mathbf{z}}$	$\mathbf{z} \leftarrow\$ \mathbb{Z}_p^l$ uniformly at random
Computations in **TEST**: (Λ, Γ)	$\xrightarrow{(\Lambda, \Gamma)}$	Checks: $\sigma := \sum_{j=0}^{l-1} \alpha^{2l-2-2j} z_j$; $E := \frac{g^{\alpha^{2l-1}}}{C}, \Delta := \frac{g^{\alpha^{2l-1}}}{\Gamma}$; 1 : $\mathsf{PoKPE}\{C, E, \Lambda, \Gamma, \Delta\}$ accepts; 2 : $C^\sigma \stackrel{?}{=} \Lambda^{\alpha^{2l}} \Gamma$

Figure 1: TEST protocol for ICP

IPP

Prover		**Verifier**
	$\xleftarrow{\mathbf{q}}$	$\mathbf{q} \in \mathbb{Z}_p^l$ is the inner product vector
Computations in **IPP**: (v, n, Λ, Γ), $N := g^n$; $(v = \langle \mathbf{f}, \mathbf{q} \rangle \mod p)$	$\xrightarrow{(v, N, \Lambda, \Gamma)}$	Checks: $\sigma := \sum_{j=0}^{l-1} \alpha^{2l-1-2j} q_j$; $\Delta := \frac{g^{\alpha^{2l-2}}}{\Gamma}$; 1 : $v \in \mathbb{Z}_p$; 2 : $\mathsf{PoKPE}\{\Lambda, \Gamma, \Delta, N\}$ accepts; 3 : $C^\sigma \stackrel{?}{=} (g^v N^p)^{\alpha^{2l-1}} \Lambda^{\alpha^{2l}} \Gamma$

Figure 2: IPP protocol for ICP

The key component of achieveing logarithmic time verifier is to use Kronecker products of log-many length-2 vectors for $\mathbf{z}$ in **TEST** and $\mathbf{q}$ in **IPP** protocols. More details on these protocols may be found on the paper [24].

Dew-PC can be made non-interactive using Fiat-Shamir transformation and the resulting scheme is succinct with respect to communication and verifier computation.

10.1 Completeness and security of Dew-PC

Theorem 5. *The polynomial commitment scheme Dew-PC satisfies completeness as defined in Definition 2.2.*

Proof. This proof was directly adopted from the original paper with little modification. The completeness of Dew-PC relies on the definition of CoeffSplit and completeness of PoKPE. Coeffsplit is a protocol used in TEST and IPP of ICP and is reproduced in Appendix B. To show that the last check in **logTEST** holds, it suffices to show that $v = 0$. Observe through direct manipulation

that

$$\begin{aligned}
&\sum_{j=0}^{l-1} f_j \alpha^{2j} \times \sum_{j=0}^{l-1} \alpha^{2l-2-2j} z_j \\
&= \alpha^{2l}\left(\sum_{j'>j} \alpha^{2(j'-j)-2} f_{j'} z_j\right) + \left(\sum_{j'<j} \alpha^{2l-2-2(j-j')} f_{j'} z_j + \sum_{j'=j} \alpha^{2l-2} f_j z_j\right) \\
&= \alpha^{2l}\lambda + \gamma.
\end{aligned}$$

The λ, γ in the end are indeed the return values of CoeffSplit and $v = 0$ as required. For **logIPP** it suffices to show that $v = f(x) \mod p$. For the query vector $\mathbf{q}$, observe that

$$\begin{aligned}
&\sum_{j=0}^{l-1} f_j \alpha^{2j} \times \sum_{j=0}^{l-1} \alpha^{2l-1-2j} q_j \\
&= \alpha^{2l-1}\left(\sum_{i=0}^{l-1} f_j q_j \mod p + p\left\lfloor \frac{\sum_{i=0}^{l-1}}{p} \right\rfloor\right) \\
&+ \alpha^{2l-2}(0) + \alpha^{2l}\left(\sum_{j'>j} \alpha^{2(j'-j)-1} f_{j'} q_j\right) + \left(\sum_{j'<j} \alpha^{2l-1-2(j-j')} f_{j'} q_j\right) \\
&= \alpha^{2l-1}\left(\sum_{i=0}^{l-1} f_j q_j \mod p + p\left\lfloor \frac{\sum_{i=0}^{l-1}}{p} \right\rfloor\right) + \alpha^{2l-2}(0) + \alpha^{2l}\lambda + \gamma.
\end{aligned}$$

The values in the end $(v + np, \lambda, \gamma)$ are indeed the output values of CoeffSplit and v is satisfied. □

The security of Dew-PC check algorithm depends on the order assumption, adaptive root assumption, and r-fractional root assumption just like DARK, under the generic group model.

11 Orion+: using a CP-SNARK

Orion+ is a multilinear commitment scheme with linear prover complexity, $O(\mu)$ proof size, and $O(\mu)$ verifier time where for μ-variate polynomials. Orion+ assists Hyperplonk [25], a multilinear-IOP, by reducing the size of the opening proof by x1000 as compared to Orion [1]. Hyperplonk is an improvement on Plonk [26], a universal fully-succinct zk-SNARK of 2022 that achieved up to 20 times lower prover time than Sonic [27], by relying on boolean hypercubes. Importantly, Orion uses a CP-SNARK called OSNARK and is therefore a CP-SNARK itself.

Definition 21 (CP-SNARK)**.** *A Commit-and-Prove SNARK (CP-SNARK) is a SNARK that aims to prove that, for vectors $\vec{f} \in \mathbb{F}^{\ltimes}$ and $\vec{g} \in \mathbb{F}^n$ with $m \leq n$ and their respective commitments C_f and C_g, the values of $\vec{f}(I_f)$ are equal to the values of $\vec{g}(I_g)$. Here, $I_f \subseteq [n]$ and $I_g \subseteq [m]$.*

Orion+ is defined by the following tuple of algorithms.

- **Setup**$(1^\lambda, \mu^*) \to gp(gp_o, gp_{pc}, vp_o, pp_o)$. On input security parameter λ, upper bound μ^* on the number of variables.
  - $m^* \leftarrow$ Set such that running time of OSNARK and PC is $O_\lambda(2^{\mu^*})$ for circuit size m^*.
  - $gp_o \leftarrow$ OSNARK.Setup$(1^\lambda, m^*)$
  - $gp_{pc} \leftarrow$ PC.Setup$(1^\lambda, m^*)$
  - $vp_o, pp_o \leftarrow$ run the indexing phase of OSNARK
- **Commit**$(\mathsf{gp}, f) \to \mathcal{C}_f$. On input polynomial $f \in \mathcal{F}_\mu^{(\leq 1)}$ with $\vec{w} = (f_{\langle 0 \rangle}, \cdots, f_{\langle n-1 \rangle})$ define the following.

- $m = n/k$
- $\vec{w} \in \mathbb{F}^{k \times m}$
- Matrix $W \in \mathbb{F}^{k \times M}$ such that $W[i,:] = E(\vec{w}[i,:]) \forall i \in [k]$ for $E : \mathbb{F}^m \to \mathbb{F}^M$ the linear encoding.
- The hash commitment $h_j \leftarrow HCom(W[:,j])$ for each $j \in [M]$, where $W[:,j]$ is the j-th column of W.
- p_h is the polynomial that interpolates vector $(h_j)_{j \in [M]}$

Output the commitment $\mathcal{C}_f \leftarrow \mathsf{PC.Commit}(gp_{pc}, p_h)$.

- **Open**(gp,$\mathcal{C}_f, f$). Run Commit and see if the output is consistent with the input $\mathcal{C}_f$.
- **Check**(gp;$\mathcal{C}_f, \vec{z}, y; f) \to \{0,1\}$. On input public parameters gp, point $\mathbf{z} \in \mathbb{F}^\mu$ and commitment $\mathcal{C}_f$, transform $\mathbf{z}$ to vectors $\vec{t_0} \in \mathbb{F}^k$ and $\vec{t_1} \in \mathbb{F}^m$ such that f($\mathbf{z}$)=$\langle \vec{w}, \vec{t_0} \otimes \vec{t_1} \rangle$. The prover and the verifier run the following protocol and return 1 on completeness, taken directly from the original paper.

1. $\mathcal{V}$ sends $\mathcal{P}$ a random vector $\mathbf{r} \in \mathbb{F}^k$.
2. Define vectors
$$\mathbf{y_r} = \sum_{i=1}^{k} \mathbf{r}_i \cdot \mathbf{w}[i,:], \qquad \mathbf{y}_0 = \sum_{i=1}^{k} \mathbf{t}_{0,i} \cdot \mathbf{w}[i,:].$$
Let $p_{\mathbf{r}}$ be the polynomial that interpolates $(\mathbf{y_r}, \mathbf{y}_0)$. $\mathcal{P}$ sends $\mathcal{V}$ commitment $C_{p_{\mathbf{y}}} \leftarrow \mathsf{PC.Commit}(\mathsf{gp}_{pc}, p_{\mathbf{y}})$.
3. $\mathcal{V}$ sends a random subset $I \subseteq [M]$ with size $t := \frac{-\lambda}{\log(1-\delta)}$.
4. $\mathcal{P}$ sends $\mathcal{V}$, a CP-SNARK proof π_o showing that
   - the statement in Figure 8 holds true;
   - the SNARK witness $(\mathbf{y_r}, \mathbf{y}_0)$ is identical to the vector committed in $C_{p_{\mathbf{y}}}$;
   - the SNARK witness $(h_j)_{j \in I}$ is consistent with that in the polynomial commitment C_f w.r.t. set I.
5. $\mathcal{V}$ checks π_o with public input $(\alpha, \mathbf{r}, y, \mathbf{z})$, and commitments $C_{p_{\mathbf{y}}}, C_f$.

Figure 3: The interactive sub-protocol for Check [25]

Observe in the Check function that Orion+ uses OSNARK to commit $(\vec{y_r}, \vec{y_0})$ to a multilinear commitment $\mathcal{C}$ and build a CP-SNARK proof for their relation, without having to directly check their consistency. This helps Orion+ reduce prover time.

11.1 Completeness and security of Orion+

Orion+ is correct from inspection. Orion+ is made possibly quantum-resistant by not relying on the discrete log problem or bilinear pairing. The security of Orion+ instead relies on that of Orion, defined in the random oracle model, and OSNARK. Additionally, Orion+ is binding and sound.

12 Appendix A

This is a generic tuple of algorithms for an inner-product commitment scheme, as presented in Vampire [12].

- $\mathsf{Setup}(1^\lambda) \to \mathsf{PGen}(1^\lambda)$.
- $\mathsf{KeyGen}(\mathsf{p}, N) \to (\mathsf{ck}, \mathsf{tk})$. On input the public parameter p and a vector dimension N, output
  - $\mathsf{ck} \leftarrow ([(\sigma^i)_{i=0:i\neq N}^{2N}]_1, [(\sigma^i)_{i=0}^{N}]_2)$
  - $\mathsf{tk} \leftarrow \sigma \xleftarrow{R} \mathbb{Z}_p^*$
- $\mathsf{Commit}(\mathsf{ck}, \mu) \to [\mu(\sigma)_1.]$ On input the commitment key ck and the vector μ that the prover wishes to commit to, return the commitment $[\mu(\sigma)_1] = \sum_{j=1}^{N} \mu_j [\sigma^j]_1$ where $\mu(X) \leftarrow \sum_{j=1}^{N} \mu_j X^j$.
- $\mathsf{Open}\ (\mathsf{ck}, [\mu(\sigma)]_1, \mu, \vec{v}) \to (v, [\mu_{ipc}(\sigma)]_1$. On input the commitment key ck, the original vector μ, another vector $\vec{v}$, compute
  - $v \leftarrow \mu^T \vec{v}$
  - $v^*(X) \leftarrow \sum_{j=1}^{N} \vec{v_j} X^{N+1-j}$
  - $\mu_{ipc}(X) \leftarrow \mu(X)\vec{v}^*(X) - vX^{N+1}$
  - $[\mu_{ipc}(\sigma)]_1 \leftarrow \sum_{i=1, i\neq N+1}^{2N} \mu_{ipc}[\sigma^i]_1$

  Finally output $(v, [\mu_{ipc}(\sigma)]_1)$.
- $\mathsf{Check}(\mathsf{ck}, [\mu(\sigma)]_1, \vec{v}, v, [\mu_{ipc}(\sigma)]_1) \to \{0, 1\}$. Output 1 if

$$[\mu_{ipc}(\sigma)]_1 \bullet [1]_2 = [\mu(\sigma)]_1 \bullet \sum_{j=1}^{N} \vec{v_j} [\sigma^{N+1-j}]_2 - v[\sigma^N]_1 \bullet [\sigma]_2$$

  and 0 otherwise.

13 Appendix B

The PoKPE protocol as used in Dew.

PoKPE (Proof of knowledge of positivity of exponent)

Params : $(\mathbb{G}, g) \xleftarrow{\$} \mathsf{GGen}(\kappa)$; Inputs : $w \in \mathbb{G}$; Witness : $x \in \mathbb{Z}$;
Claim : $(g^x = w) \wedge (x > 0)$

1. Prover sets $x = a_1^2 + a_2^2 + a_3^2 + a_4^2 \in \mathbb{Z}$ and $P_i := g^{a_i^2}$.
2. Prover and Verifer engage in PoKSE for each P_i.
3. Verifier accepts if all the PoKSE's output `accept` and at least one $P_i \neq 1$ and if $w = P_1 P_2 P_3 P_4$.

Figure 4: Protocol PoKPE [24]

The CoeffSplit algorithm is defined as follows in Dew.

$\mathsf{CoeffSplit}(\alpha, a, b, i)$

1. Write $a \cdot b$ in base α, call the resulting representation vector $\mathbf{c}$.
2. Set $v := c_i$, $\gamma := \sum_{j=0}^{i-1} c_j \alpha^j$, $\lambda := \sum_{j=i+1}^{\lceil \log_\alpha ab \rceil} c_j \alpha^j$
3. Output the tuple (v, γ, λ).

Figure 5: CoeffSplit algorithm [24]

References

[1] T. Xie, Y. Zhang, and D. Song, "Orion: Zero knowledge proof with linear prover time," Cryptology ePrint Archive, Paper 2022/1010, 2022, https://eprint.iacr.org/2022/1010. [Online]. Available: https://eprint.iacr.org/2022/1010

[2] D. Khurana, "One way functions, hard core bit - i," 2018. [Online]. Available: https://courses.grainger.illinois.edu/cs598dk/fa2019/Files/lecture02.pdf

[3] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," Cryptology ePrint Archive, Paper 2017/1066, 2017, https://eprint.iacr.org/2017/1066. [Online]. Available: https://eprint.iacr.org/2017/1066

[4] J. Bootle, A. Chiesa, and J. Groth, "Linear-time arguments with sublinear verification from tensor codes," in *Theory of Cryptography: 18th International Conference, TCC 2020, Durham, NC, USA, November 16–19, 2020, Proceedings, Part II 18*. Springer, 2020, pp. 19–46.

[5] N. Ron-Zewi and R. D. Rothblum, "Local proofs approaching the witness length [extended abstract]," *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS)*, 2020.

[6] A. Golovnev, J. Lee, S. Setty, J. Thaler, and R. S. Wahby, "Brakedown: Linear-time and post-quantum snarks for r1cs," Cryptology ePrint Archive, Paper 2021/1043, 2021, https://eprint.iacr.org/2021/1043. [Online]. Available: https://eprint.iacr.org/2021/1043

[7] J. Lee, "Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments," Cryptology ePrint Archive, Paper 2020/1274, 2020, https://eprint.iacr.org/2020/1274. [Online]. Available: https://eprint.iacr.org/2020/1274

[8] J. Bootle, A. Cerulli, P. Chaidos, J. Groth, and C. Petit, "Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting," *Advances in Cryptology – EUROCRYPT 2016*, p. 327–357, 2016.

[9] J. Groth, "Efficient zero-knowledge arguments from two-tiered homomorphic commitments," *Lecture Notes in Computer Science*, p. 431–448, 2011.

[10] M. Abe, G. Fuchsbauer, J. Groth, K. Haralambiev, and M. Ohkubo, "Structure-preserving signatures and commitments to group elements," in *Advances in Cryptology–CRYPTO 2010: 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010. Proceedings 30*. Springer, 2010, pp. 209–236.

[11] [Online]. Available: https://www.doc.ic.ac.uk/~mrh/330tutor/ch06s02.html

[12] H. Lipmaa, J. Siim, and M. Zajac, "Counting vampires: From univariate sumcheck to updatable zk-snark," Cryptology ePrint Archive, Paper 2022/406, 2022, https://eprint.iacr.org/2022/406. [Online]. Available: https://eprint.iacr.org/2022/406

[13] J. Bootle, A. Chiesa, and K. Sotiraki, "Sumcheck arguments and their applications," Cryptology ePrint Archive, Paper 2021/333, 2021, https://eprint.iacr.org/2021/333. [Online]. Available: https://eprint.iacr.org/2021/333

[14] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward, "Aurora: Transparent succinct arguments for r1cs," Cryptology ePrint Archive, Paper 2018/828, 2018, https://eprint.iacr.org/2018/828. [Online]. Available: https://eprint.iacr.org/2018/828

[15] M. Izabachène, B. Libert, and D. Vergnaud, "Block-wise p-signatures and non-interactive anonymous credentials with efficient attributes," in *Cryptography and Coding*, 12 2011, pp. 431–450.

[16] C. Papamanthou, E. Shi, and R. Tamassia, "Signatures of correct computation," *Theory of Cryptography*, p. 222–242, 2013.

[17] R. L. A. Chiesa, "Multivariate commitments for a single degree bound," *not applicable*, unknown.

[18] A. Kate, G. M. Zaverucha, and I. Goldberg, "Constant-size commitments to polynomials and their applications," in *Advances in Cryptology-ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings 16*. Springer, 2010, pp. 177–194.

[19] D. Boneh and X. Boyen, "Short signatures without random oracles and the SDH assumption in bilinear groups," *J. Cryptol.*, vol. 21, no. 2, pp. 149–177, 2008. [Online]. Available: https://doi.org/10.1007/s00145-007-9005-7

[20] B. Bünz, B. Fisch, and A. Szepieniec, "Transparent snarks from dark compilers," Cryptology ePrint Archive, Paper 2019/1229, 2019, https://eprint.iacr.org/2019/1229. [Online]. Available: https://eprint.iacr.org/2019/1229

[21] K. Pietrzak, "Simple verifiable delay functions," in *10th innovations in theoretical computer science conference (itcs 2019)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2018.

[22] D. Boneh, B. Bünz, and B. Fisch, "Batching techniques for accumulators with applications to iops and stateless blockchains," in *Advances in Cryptology – CRYPTO 2019*, A. Boldyreva and D. Micciancio, Eds. Cham: Springer International Publishing, 2019, pp. 561–586.

[23] B. Wesolowski, "Efficient verifiable delay functions," *J. Cryptol.*, vol. 33, no. 4, p. 2113–2147, oct 2020. [Online]. Available: https://doi.org/10.1007/s00145-020-09364-x

[24] A. Arun, C. Ganesh, S. Lokam, T. Mopuri, and S. Sridhar, "Dew: Transparent constant-sized zksnarks," Cryptology ePrint Archive, Paper 2022/419, 2022, https://eprint.iacr.org/2022/419. [Online]. Available: https://eprint.iacr.org/2022/419

[25] B. Chen, B. Bünz, D. Boneh, and Z. Zhang, "Hyperplonk: Plonk with linear-time prover and high-degree custom gates," Cryptology ePrint Archive, Paper 2022/1355, 2022, https://eprint.iacr.org/2022/1355. [Online]. Available: https://eprint.iacr.org/2022/1355

[26] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge," Cryptology ePrint Archive, Paper 2019/953, 2019, https://eprint.iacr.org/2019/953. [Online]. Available: https://eprint.iacr.org/2019/953

[27] M. Maller, S. Bowe, M. Kohlweiss, and S. Meiklejohn, "Sonic: Zero-knowledge snarks from linear-size universal and updateable structured reference strings," Cryptology ePrint Archive, Paper 2019/099, 2019, https://eprint.iacr.org/2019/099. [Online]. Available: https://eprint.iacr.org/2019/099